AILT9019 · AI Literacy II

Project Proposal Template

Due: Friday, 25 Sep (Week 4) · One submission per team, via the Moodle, TEAM 34

1. Background

Who is this for, what is stuck, why it matters this semester, and who or what it is not for. Also state where the idea comes from: which topic-list idea(s) and how far they were adapted, an external idea, or an original idea. A figure is optional.

Who is this for

New HKU students, exchange students, and staff are unfamiliar with campus eateries. Especially those with short breaks between classes.

What is stuck

HKU campus is complex; restaurants are scattered with different hours. Current tools (CEDARS list, maps) are fragmented and can't filter by needs like "spicy, cheap, open now".

Why it matters this semester

We must build a working AI app from zero. This real campus problem lets us practice LLM, rule filtering, and maps. It fits course goals: use AI tools, solve real problems, think about ethics.

Who or what it is not for

Not for outsiders. Not for delivery/booking. Not for reviews. Not an indoor GPS. Only for HKU campus dining recommendations and walking directions between buildings.

Where the idea comes from

Original idea from our team's experience as HKU students: got lost finding food, canteens closed. Classmates often ask "where's veg?" or "what's open?". So, we built an AI tool to help.

2. Project pipeline

End-to-end path: request → method → result, and what the team will actually build. A pipeline diagram is encouraged.

Input:

The user starts or continues a conversation in natural language (e.g., "spicy, cheap, open now" or "I'm at the Main Library"). The system supports multi-turn dialogue and can read previous messages in the current session.

Method:

***Dialogue management**: The system maintains session context, including past preferences, current building, diet restrictions, and time constraints.*

***Information extraction**: An LLM parses the latest user message and merges the extracted filters (cuisine, price, diet, time, location) with the existing context.*

***Proactive questioning**: If critical information is missing (especially the user's current building), the system asks 1–2 clear follow-up questions instead of forcing a recommendation.*

***Rule-based filtering**: Using the fixed HKU canteen dataset (name, building, floor, opening hours, price range, diet tags), the system keeps only canteens that are currently open, match the diet/price needs, and are within reasonable walking range.*

***Ranking & explanation**: The LLM ranks the remaining options and writes a short reason for each.*

***Navigation**: Simple walking direction and estimated time from the user's stated building to the recommended canteen are provided.*

***Continued dialogue**: After seeing the results, the user can refine the request (e.g., "make it vegetarian" or "something cheaper"), and the system updates the recommendations based on the new context.*

Output:

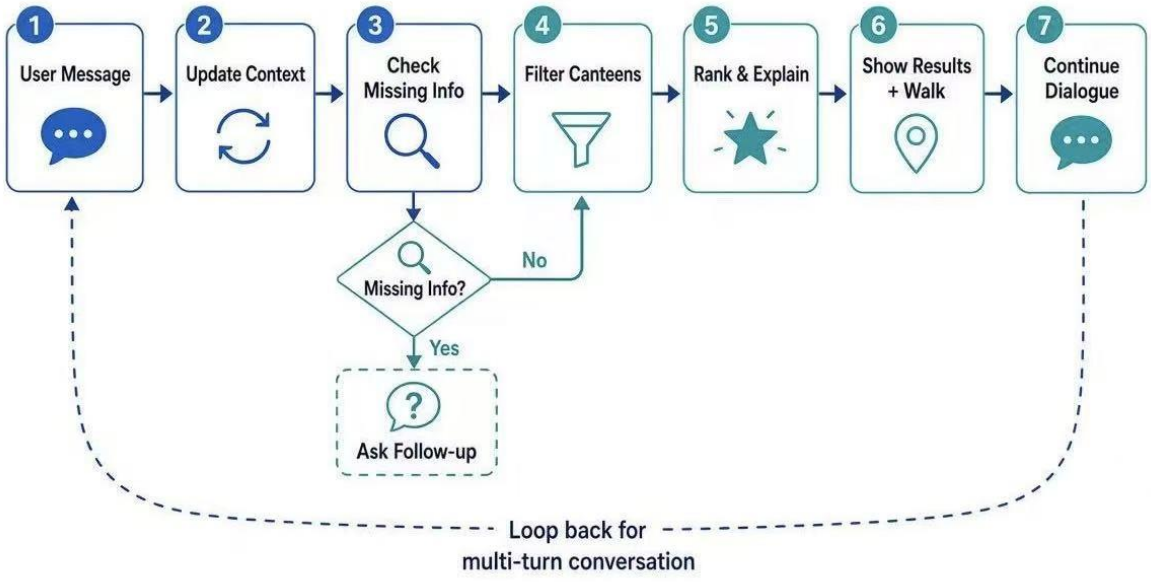
Top 2–3 canteens, each showing: name, building/floor, why it fits, walk time, and a one-line tip. The conversation can continue.

What the team will actually build:

A simple browser or local conversational app with:

- *a fixed, self-curated HKU canteen dataset*
- *session-level context memory*
- *an LLM for understanding requests, merging context, and generating reasons*
- *rule filters for opening hours and diet tags*
- *ability to proactively ask for missing critical information*
- *building-to-building walking information*

Not included: Delivery service, table booking, off-campus restaurants, indoor GPS, or long-term user profiling.



3. Evaluation and test

How the team will tell the product works, including at least one case it should fail safely or refuse.

Goal

Verify that the system can accurately understand natural-language dining requests, maintain multi-turn context, proactively ask for missing critical information, correctly filter the fixed canteen dataset, return useful recommendations with reasons and walking directions, and safely refuse impossible or out-of-scope requests.

Test set design (approximately 18-20 questions)

A. Normal successful cases (10–12 questions):

These cover common real-life student requests. For example:

• “Spicy, cheap, and open now”

• “Vegetarian options near the Main Library”

• “I only have 25 minutes between classes – what’s quick?”

• “Not too expensive, want noodles”

• “Somewhere quiet for coffee”

Each question specifies cuisine/diet preference, price sensitivity, and the user’s current building.

B. Required refusal cases (2-3 questions)

1. Unsatisfiable request Example: “still open at 11 pm” or “High-end lobster restaurant on campus” Expected behavior: The system clearly states that no matching canteen exists in the current dataset and suggests adjusting the request.

2. Out-of-scope request Example: “Deliver food to my dorm” / “Book a table for me” / “Recommend a restaurant outside campus” Expected behavior: The system politely refuses and explains that the tool only provides on-campus canteen recommendations and walking directions.

C. Edge / ambiguous cases (2–3 questions)

• Conflicting requirements (e.g., “very cheap and very high-end”)

• Only location given, no preference (“I’m at Centennial Campus, what can I eat?”)

• Vague diet description (“something healthy”)

D. Multi-turn dialogue & proactive questioning cases (a long chatbot including 3-4 questions)

1. User only says "I want something cheap" System should proactively ask for current building or time constraint.
2. User first says "I'm at the Main Library", then "Any vegetarian options?" System should combine context and give recommendations.
3. After receiving results, user says "Make it cheaper" or "Only vegetarian" > System should update recommendations based on previous context.

4. If user never provides a building after being asked once or twice, the system should either give general (location-independent) suggestions or politely explain that building information is needed for walking directions.

Pass criteria

A case is considered passed only when all relevant points below are satisfied:

- Parsing & context: Correctly extracts and merges information across turns.
- Proactive asking: When critical information (especially current building) is missing, the system asks a clear follow-up question instead of guessing or failing silently.
- Filtering: Only returns currently open canteens that match the stated constraints.
- Recommendation quality: Returns 2-3 options, each with a short reason and walking information (when location is known).
- Refusal: Impossible or out-of-scope requests are clearly and politely refused.
- Context update: When the user modifies a previous request, the new results reflect the change.

How we will run the evaluation

Evaluation process

- The full test set will be run manually after each major development checkpoint.
- Mid-term target: 90% pass rate on normal cases, 100% correct handling of refusal cases, and successful proactive questioning in the multi-turn cases.
- Final demonstrations will include both single-turn and multi-turn examples.
- Recording method: A simple table will be kept ("Question → System output → Pass/Fail → Notes") for reproducibility and peer review.

4. Distribution and accessibility

How someone else can open the app: API setup, first use without a coding background, and (if in scope) other agent environments.

1. A user opens a simple web page or small local app, sets their preferences (taste, budget, spice level, vegetarian, etc.), and then views recommendations.

2. No coding background is needed for first use: the README explains three steps — open the app, set preferences, view recommendations.

3.For the model, a lightweight local model or small API is used so users do not have to configure a complex environment; if an API key is required, the README states where to get it and where to enter it.

4.If the API is unavailable, cached recommendations or a rule-based fallback keep the demo usable.

5.If in scope, the recommendation logic can be packaged as a skill file, so other agent environments can call the same preference-based recommendation capability.

5. Ethics and risks

An explicit "does not", plus main misuse or failure risks and how they are mitigated.

Does not:

1. *This tool does not provide food delivery or table booking services*
2. *It does not recommend off-campus restaurants*
3. *It does not use or store long-term personal data or conversation history beyond the current session*
4. *It does not claim real-time accuracy of opening hours or menus (the dataset is a fixed snapshot)*
5. *It does not give medical or allergy advice beyond simple diet-tag matching*

Main risks:

1. *Outdated information in the fixed dataset (opening hours or diet tags) may lead a student to a closed or unsuitable canteen.*
2. *The system may misunderstand vague or incomplete requests, resulting in less relevant recommendations.*
3. *Users might over-rely on the tool and skip checking the actual situation on site*

Mitigation measures:

1. *A clear on-screen notice states that all information is based on a fixed snapshot and users should always verify opening hours and details on site.*
2. *The system proactively asks for missing critical information (especially current building) instead of guessing.*
3. *Out-of-scope requests (delivery, booking, off-campus) and clearly impossible requests are refused with a short explanation.*
4. *Only session-level context is kept; no long-term user profiles or personal data are stored.*
5. *Diet matching is limited to the tags available in the dataset and is presented as a reference only.*

6. Checkpoints

Must-have and should-have acceptance items, plus a timeline to the Week 4 draft. A timeline figure is encouraged.

Must-have (the system is only "done" when all of these pass)

1. Canteen dataset: A fixed, self-curated dataset of HKU Main Campus canteens built from public information. Each entry has name, building, floor, opening hours, price range, cuisine and diet tags, plus a "last updated" date.

2. Multi-turn conversations and context: The system supports multi-turn conversations and retains information about the context of the conversation, such as any preferences mentioned, the current building, dietary restrictions and time constraints.

3. Request parsing & proactive questioning: The LLM turns natural-language requests into structured filters (cuisine, price, diet, time). When critical information is missing (especially the user's current building), the system proactively asks 1–2 clear follow-up questions instead of forcing a recommendation.

4. Rule filter: Code, not the LLM, removes canteens that are closed at the given time or that do not match the diet or price constraints. No closed or mismatched canteen may appear in any output.
5. Ranking and reasons: The LLM ranks the remaining options and returns the top 2–3 canteens, each with a one-line reason.
6. Walking time: A self-curated building-to-building walk-time table gives the walk time and a simple direction from the user's building to each recommended canteen.
7. Safe refusal & boundary: The system clearly explains when no canteen matches (unsatisfiable requests), proactive questioning occurs when critical info like user building is missing, and politely refuses out-of-scope requests (delivery, booking, off-campus) or medical/allergy questions.
8. Test threshold: The fixed test set from Section 3 is passed at the mid-term check with at least 90% pass rate on normal cases and 100% on refusal cases, recorded in the Question → System output → Pass/Fail → Notes table.

Should-have (only if the must-have is finished and passing)

1. Voice input in addition to typed input.
2. A rule-based fallback when the LLM API is unavailable, so the demo still runs.
3. A simple map view of the walking route instead of text directions only.
4. Coverage of Centennial Campus buildings beyond the Main Campus core.
5. Packaging the recommendation logic as a skill file for other agent environments.

Timeline

Week	Phase / Status	Key Checkpoint & Objectives	Deliverable / Milestone
W2–3	**● Planning**	Topic shortlisted; HKU canteen recommendation idea selected.	Shortlist done
W4	**● Proposal**	Project scope fixed; initial proposal draft completed for Team 34.	Draft submitted (Fri 25 Sep)
W5	**● Data & Review**	HKU canteen dataset and building-to-building walk-time table collected; peer reviews written.	Peer reviews due (Mon 5 Oct)
W6	**● Development**	LLM request parsing module functional;	Revised

		initial test-set run recorded; proposal revised based on peer feedback.	proposal (Sat 10 Oct)
W7	**● Integration**	Rule-based filter (hours, diet, price) and walk-time lookup module integrated into pipeline.	Second test-set run recorded
W8	**● Pipeline Sync**	LLM ranking & reason generation connected for end-to-end flow; proactive questioning validated; proposal consultation completed.	Consultation completed
W9	**● Evaluation**	End-to-end demo evaluated; test set target met (≥90% pass rate on normal cases, 100% on refusal cases, proactive questioning working).	Mid-term check (Mon 26 Oct)
W10–11	**● Refinement**	Fix failed cases, refine edge-case handling, and publish README with 3-step setup guide. Explore should-have features if must-haves are stable.	README & setup guide ready
W12	**● Code Freeze**	Must-have capabilities frozen; final full test-set run conducted and logged in evaluation table.	Complete results table
W13	**● Final Demo**	Final live demonstration featuring 4 random normal cases and 2 refusal cases.	Final live demo